

THE ONE-PAGE-PER-PATTERN GUIDE

# Coding Interview Patterns

20 patterns that solve most interview problems,  
each on a single page with tested Python code.



```
$ python test_snippets.py  
all snippet tests passed
```

# How to use this guide

Interview problems look endless, but most of them are the same 20 or so ideas in different clothes. Once you can spot the **pattern**, the problem becomes a template plus a few details.

Every pattern gets exactly one page:

- **Use it when:** the clues in a problem statement that point to this pattern.
- **The idea** in plain words, then a short **template** in Python. Every snippet in this book is run against tests before publishing.
- **Watch out:** the bugs interviewers see most often.
- **Practice:** four well-known problems (search the names on LeetCode or similar sites), from easier to harder.
- **Say it in the interview:** one sentence that shows you understand *why* it works.

## A GOOD WAY TO STUDY

Read one pattern a day. Close the book and write the template from memory. Then solve the first two practice problems without hints, and the last two with hints if you need them. The 14-day plan at the back does exactly this.

## Big-O in one minute

| INPUT SIZE N           | USUALLY NEEDS           | TYPICAL PATTERN                  |
|------------------------|-------------------------|----------------------------------|
| $n \leq 20$            | $O(2^n)$ or $O(n!)$     | Backtracking                     |
| $n \leq 500$           | $O(n^3)$                | Small DP tables                  |
| $n \leq 5,000$         | $O(n^2)$                | DP, nested loops                 |
| $n \leq 10^6$          | $O(n \log n)$ or $O(n)$ | Sort, heap, two pointers, window |
| Huge values ( $10^9$ ) | $O(\log n)$             | Binary search                    |

© 2026 Mahdi Hasan Shadi / PageNook. For personal use; please don't redistribute. Problem names are mentioned for practice only; they belong to their respective platforms. Written with the help of AI tools and checked by running every code sample.



# Which pattern is it?

*Read the problem, find the clue on the left, and start with the pattern on the right.*

| IF THE PROBLEM SAYS OR NEEDS...      | TRY                          |
|--------------------------------------|------------------------------|
| Sorted array, find a pair            | Two Pointers / Binary Search |
| Longest / shortest contiguous range  | Sliding Window               |
| Subarray sums with negatives         | Prefix Sums                  |
| “Have I seen this?”, pairs, grouping | Hash Maps                    |
| Linked list cycle or middle          | Fast & Slow Pointers         |
| “Minimum X such that...”             | Binary Search on the Answer  |
| Next greater / smaller element       | Monotonic Stack              |
| Overlapping time ranges              | Intervals                    |
| Shortest path, equal step costs      | BFS                          |
| Shortest path, weighted steps        | Dijkstra                     |
| Explore connected regions            | DFS / Union-Find             |
| Dependencies and order               | Topological Sort             |
| All combinations / arrangements      | Backtracking                 |
| Top k, kth largest, k closest        | Heaps                        |
| Count ways / best value with choices | Dynamic Programming          |
| Prefix lookups over many words       | Tries                        |
| Pairs cancel, bit counts             | Bit Manipulation             |

## STILL STUCK?

Write the brute force first and say its complexity. Then ask: what work is it repeating? Removing repeated work is what most patterns do.

# 01 Two Pointers

*Walk inward from both ends of a sorted array instead of checking every pair.*

## USE IT WHEN

- The input is sorted (or you can sort it)
- You need a pair or triplet that meets a target
- Palindromes, reversing, or partitioning in place

## The idea

Start one pointer at each end. If the sum is too small, move the left pointer right to make it bigger; if it's too big, move the right pointer left. Every step rules out a whole row of pairs, so one pass is enough.

## Template

```
def pair_with_sum(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target:
            return lo, hi
        if s < target:
            lo += 1
        else:
            hi -= 1
    return None
```

**Cost:**  $O(n)$  time,  $O(1)$  space (plus  $O(n \log n)$  if you sort first)

## WATCH OUT

- It only works because the array is sorted. Say so out loud.
- For 3Sum, fix one number and run two pointers on the rest; skip equal neighbours to avoid duplicate answers.

## PRACTICE

- Two Sum II
- Valid Palindrome
- 3Sum
- Container With Most Water

**Say it in the interview:** “Because it's sorted, if the sum is too small the only way to grow it is to move the left pointer.”

## Sliding Window

**02** *Keep a window over a subarray or substring, growing the right edge and shrinking the left.*

### USE IT WHEN

- “Longest / shortest substring or subarray that...”
- A condition over a contiguous range
- Brute force would re-scan overlapping ranges

### The idea

Move the right edge forward one step at a time. When the window breaks the rule (a repeated letter, a sum too large), move the left edge until it's valid again. Each index enters and leaves the window at most once.

### Template

```
def longest_unique_substring(s):
    last = {}          # char → last index seen
    start = best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] ≥ start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best
```

**Cost:**  $O(n)$  time,  $O(k)$  space for the window's contents

### WATCH OUT

- Sum-based windows need non-negative numbers. With negatives, switch to prefix sums.
- Update the answer at the right moment: after fixing the window, not before.

### PRACTICE

- Longest Substring Without Repeating Characters
- Minimum Size Subarray Sum
- Longest Repeating Character Replacement
- Permutation in String

**Say it in the interview:** “Each character is added once and removed at most once, so this is linear.”